

Operating System Lab Assignment – 4



Implementation and Analysis of Disk Scheduling Algorithms

Submitted By:

Tarun
BCA (AI & Data Science) - B
2401201077

1. Introduction

Disk scheduling is an essential function of an operating system that determines the order in which disk I/O requests are serviced. Since disk access time depends largely on the movement of the disk head, efficient scheduling algorithms are required to minimize seek time and improve overall system performance.

In this assignment, various disk scheduling algorithms such as FCFS, SSTF, SCAN, and C-SCAN are implemented and analyzed based on total head movement (seek time).

2. Objectives

- To understand disk scheduling concepts
 - To implement different disk scheduling algorithms
 - To calculate total seek time
 - To compare the performance of each algorithm
-

3. Tools & Technologies Used

- Python 3.x
 - VS Code / PyCharm
 - Ubuntu/Linux Terminal
 - Basic Python Libraries
-

4. Input Details

- Disk Request Queue: (example: 98, 183, 37, 122, 14, 124, 65, 67)
 - Initial Head Position: (example: 53)
 - Disk Size: (example: 200)
-

5. Algorithms Implemented

5.1 FCFS (First Come First Serve)

This algorithm services requests in the order they arrive.

Advantages:

- Simple and easy to implement

Disadvantages:

- High seek time
 - Poor performance
-

5.2 SSTF (Shortest Seek Time First)

This algorithm selects the request closest to the current head position.

Advantages:

- Reduces seek time
- Better than FCFS

Disadvantages:

- May cause starvation
-

5.3 SCAN (Elevator Algorithm)

The disk head moves in one direction, servicing requests, and then reverses direction.

Advantages:

- Balanced performance
- Less starvation

Disadvantages:

- Slightly complex
-

5.4 C-SCAN (Circular SCAN)

The disk head moves in one direction only and jumps back to the beginning.

Advantages:

- Uniform waiting time
- Fair scheduling

Disadvantages:

- Extra movement due to jump
-

6. Output Results

Algorithm Total Seek Time

FCFS	XX
SSTF	XX
SCAN	XX
C-SCAN	XX

(Replace XX with your actual output values)

7. Performance Comparison

- SSTF generally gives the lowest seek time
 - FCFS has the highest seek time
 - SCAN provides balanced performance
 - C-SCAN ensures fairness and uniform wait time
-

8. Result Analysis

From the observed results, SSTF performs better in minimizing seek time as it always selects the nearest request. However, it may lead to starvation for some requests.

SCAN and C-SCAN provide a more balanced approach by servicing requests in an ordered manner. C-SCAN improves fairness by treating all requests uniformly.

FCFS, although simple, is not efficient due to large head movements.

9. Conclusion

This assignment provided a clear understanding of disk scheduling algorithms and their impact on system performance. It highlighted how different approaches affect total seek time.

Among all algorithms, SSTF is efficient in reducing seek time, while SCAN and C-SCAN offer better fairness and balanced performance.

10. References

- Operating System Concepts by Silberschatz
 - Lecture Notes
 - Python Documentation
-